Git Fundamentals

Module 3 · Session cheat sheet · Concise reference for the lab

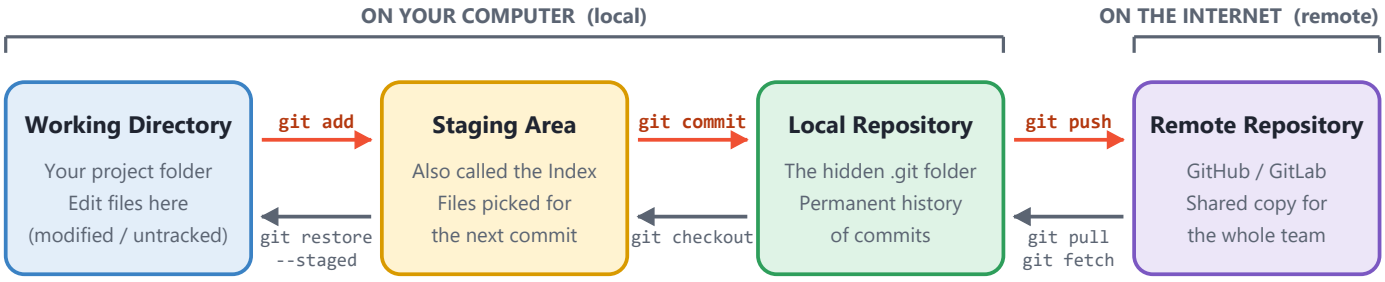
DAY 1

WHY VERSION CONTROL?

History	Undo	Collaborate
See who changed what, when and why	Roll back to any earlier version safely	Many people, one project, no overwriting

1 Git Architecture *the three areas a file travels through*

Git is a **distributed** version control system: every developer keeps the full project history on their own computer.



Working Directory	The files you see and edit. Git notices changes but is **not saving** them yet.
Staging Area	A holding zone for the **next commit**. You choose exactly which changes go in with `git add`.
Local Repository	The **.git** folder: permanent snapshots (commits) stored on your computer. `git commit` writes here.
Remote Repository	A copy hosted online (GitHub) for backup and sharing. `git push` sends commits there.

Parcel analogy: items on your table (working directory) → put into the box (staging) → seal and label the box (commit + message) → stored in the warehouse (repository).

Under the hood

- **Commit** = snapshot + message + author + unique ID (SHA) + link to its parent
- **Branch** = a lightweight, movable pointer to a commit
- **HEAD** = pointer to where you are right now

Git vs GitHub

- **Git** = the tool on your computer that tracks changes (works offline)
- **GitHub** = a website that hosts Git repositories for sharing and teamwork
- Alternatives to GitHub: GitLab, Bitbucket

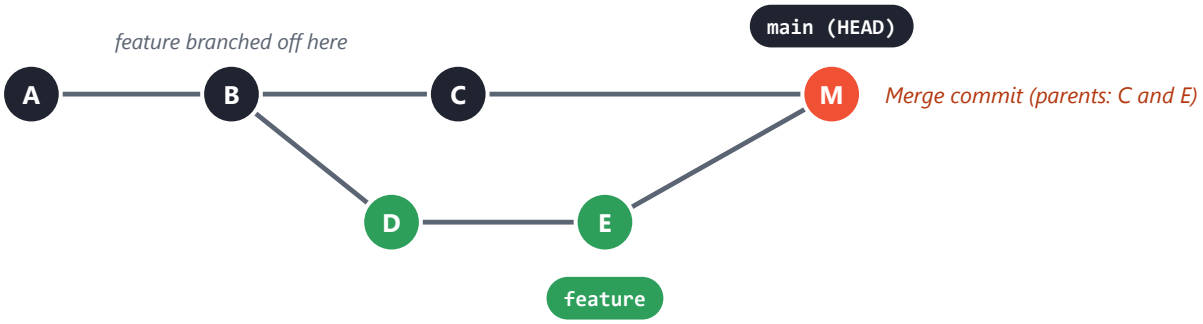
2 Command Cheat Sheet *Git Lab commands are tagged LAB*

SETUP & START		
	`git config --global user.name "Name"`	Set who you are (also **user.email**); stamped on every commit
LAB	`git init`	Turn the current folder into a Git repository (creates **.git**)
	`git clone <url>`	Copy an existing remote repository to your computer
DAILY WORKFLOW		
LAB	`git status`	Show which files are modified, staged or untracked
LAB	`git add <file> | git add .`	Move changes to the **staging area** (one file / everything)
LAB	`git commit -m "message"`	Save the staged snapshot permanently in the **repository**
LAB	`git log --oneline --graph`	Show commit history, one line per commit, with branch graph
	`git diff`	See what changed but is not staged yet
BRANCHING & MERGING		
LAB	`git branch`	List branches (* marks the current one)
LAB	`git branch <name>`	Create a new branch
LAB	`git checkout <name>`	Switch to that branch (newer alias: **git switch**)
	`git checkout -b <name>`	Create a branch and switch to it in one step
	`git merge <name>`	Merge that branch into the branch you are currently on
	`git branch -d <name>`	Delete a branch after it has been merged
REMOTE (GITHUB)		
	`git remote add origin <url>`	Link your local repository to a GitHub repository
	`git push -u origin main`	Upload your commits to GitHub
	`git pull`	Download the latest commits and merge them in
UNDO		
	`git restore --staged <file>`	Unstage a file (changes stay in your working directory)
	`git restore <file>`	Discard unsaved edits in a file – **cannot be undone**

Typical flow: `git init` → `git add .` → `git commit -m "msg"` → `git branch` / `git checkout` → `git merge`

<angle brackets> = replace with your own value. The default branch is called **main** (**master** in older Git versions).

3 Merging Branches *step by step*



1	`git status`	Working tree must be clean. Commit your pending work first.
2	`git checkout main`	Switch to the branch that should **receive** the changes.
3	`git merge feature`	Bring the commits of **feature** into **main**.
4	`git log --oneline --graph`	Check the result. **Fast-forward** or **Merge made by...** = success. **CONFLICT** = go to section 4.
5	`git branch -d feature`	Optional: delete the branch once it is merged.

Fast-forward: if **main** has no new commits since **feature** branched off, Git just moves the pointer forward – no merge commit is created.

4 Resolving a Merge Conflict *when Git cannot decide for you*

A conflict happens when **both branches changed the same lines** of the same file. Git pauses the merge and marks the clash inside the file:

```
<<<<<<< HEAD                          ← start of your version (current branch)
<h1>Welcome to my portfolio</h1>      ← your branch (main)
=======                               ← divider
<h1>Hi, I'm a CS student!</h1>        ← incoming branch (feature)
>>>>>>> feature                       ← end of incoming version
```

1	`git merge feature`	Git stops: *CONFLICT (content): Merge conflict in index.html*. The merge is paused, not broken.
2	`git status`	Files under **Unmerged paths** are the ones to fix.
3	**Open the file**	Find the markers and decide: keep yours, keep theirs, or **combine both**.
4	**Edit and clean up**	Delete the 3 marker lines (`<<<<<<<`, `=======`, `>>>>>>>`), keep only the final code, save.
5	`git add index.html`	Tells Git this file is **resolved** (stages it).
6	`git commit -m "Merge feature"`	Completes the merge. Verify with `git log --oneline --graph`.

Changed your mind?
`git merge --abort` returns to the state before the merge.

Avoid conflicts: commit small and often, `git pull` before you push, and don't edit the same lines on two branches.